Cambiar a PokeAPI sobre el código que ya tienes

Sin borrar archivos ni reescribir todo: se modifica lo que existe. Cada línea sale de algo que **puedes ver**: el JSON, tu código viejo o las entidades.

Las 3 fuentes: de aquí sale todo

Fuente	Dónde la ves	Qué te da
El JSON	Navegador: `https://pokeapi.co/api/v2/pokemon/ditto` (marca "Dar formato al texto")	Los nombres entre comillas y los tipos de los datos
Tu código viejo	`ClimaDtos.kt` y `ClimaMapper.kt`	El **molde**: cómo se escribe cada cosa
Las entidades	Ctrl + clic en `Ubicacion`, `ClimaActual`, `Condicion`…	Los nombres del lado izquierdo del mapper

Cómo viaja un dato

```
JSON:      "name": "ditto"
             |
DTO:       @SerializedName("name") val nombre: String          → nombre = "ditto"
             |
Mapper:    name = nombre                                       → Ubicacion.name = "ditto"
             |
Pantalla:  título grande "ditto"  (donde antes decía "La Paz")
```

Los 5 cambios (y qué queda igual)

Paso	Archivo	Qué cambias
1	`apikey.properties` (nuevo, en la raíz)	Una línea con una clave cualquiera
2	`datos/ApiClient.kt`	1 línea: la URL base
3	`datos/WeatherApiService.kt`	3 líneas: `@GET`, `@Path` y un import
4	`datos/dto/ClimaDtos.kt`	Solo `RespuestaClimaDto` y una clase nueva `SpritesDto`
5	`datos/mapper/ClimaMapper.kt`	Solo la **última** función

No se tocan: `ClimaRepository`, `ClimaViewModel`, las pantallas, las entidades, ni las demás clases y funciones de los DTOs y del mapper.

> **Respaldo**
> En `respaldo_weatherapi/` están los 4 archivos originales. Si algo se rompe, copia de ahí la parte original.

Paso 0 · Leer el JSON

☐ Abre `https://pokeapi.co/api/v2/pokemon/ditto` en Chrome.

☐ Marca la casilla **"Dar formato al texto"** (arriba a la izquierda). También puedes pegarlo en `jsonformatter.org`.

☐ Usa **Ctrl + F** para encontrar cada campo: `"height"`, `"weight"`...

> **Usa la URL correcta**
> `.../pokemon/ditto` devuelve un **Pokémon**. `.../type/fire` devuelve un **tipo**, que no tiene `height` ni `weight`. Cada URL tiene su propio JSON.

Los campos que usaremos están en el **primer nivel**, es decir, sueltos y no dentro de otra cosa:

```
{
  "id": 132,
  "name": "ditto",
  "height": 3,
  "weight": 40,
  "sprites": {
    "front_default": "https://raw.githubusercontent.com/.../132.png",
    ...
  },
  "moves": [ ... muy largo ... ],
  ...
}
```

Regla de tipos: mira qué hay después de los dos puntos

En el JSON ves	Ejemplo	Tipo en Kotlin
Texto con comillas	`"ditto"`	`String`
Número sin decimales	`132`	`Int`
Número con decimales	`1.5`	`Double`
`true` / `false`	`true`	`Boolean`
Llaves `{ }`	`"sprites": { ... }`	**Una `data class` nueva**
Corchetes `[ ]`	`"types": [ ... ]`	`List<...>`
Puede venir `null`	`"front_default": null`	Tipo con `?` (`String?`)

Paso 1 · La clave

PokeAPI **no usa clave**, pero tu código sí la exige: `ClimaRepository` dice "Falta la clave" y `ClimaViewModel` activa el modo demo si está vacía. Para no tocar esos archivos, le das una clave cualquiera.

☐ En la carpeta raíz `Api-main` (donde está `apikey.properties.example`), crea el archivo **`apikey.properties`**.

☐ Escribe dentro: `WEATHER_API_KEY=pokeapi`

☐ **File → Sync Project with Gradle Files.**

Paso 2 · La URL base

✓ ya lo hiciste

datos/ApiClient.kt

```
.baseUrl("https://pokeapi.co/api/v2/")      // siempre termina en "/"
```

De dónde sale

De la URL de ejemplo: `https://pokeapi.co/api/v2/` **pokemon/ditto**. Lo que va **antes** de `pokemon/` es la URL base.

Paso 3 · La ruta y el nombre

antes

```
import retrofit2.http.GET
import retrofit2.http.Query

interface WeatherApiService {
    @GET("forecast.json")
    suspend fun obtenerPronostico(
        @Query("key") key: String,
        @Query("q") ciudad: String,
        @Query("days") dias: Int = 3,
        @Query("lang") idioma: String = "es",
        @Query("aqi") calidadAire: String = "no",
        @Query("alerts") alertas: String = "no"
    ): RespuestaClimaDto
}
```

después

```
import retrofit2.http.GET
import retrofit2.http.Header                    // ← en lugar de Query
import retrofit2.http.Path                      // ← nuevo

interface WeatherApiService {
    @GET("pokemon/{nombre}")                    // ← antes: "forecast.json"
    suspend fun obtenerPronostico(
        @Header("key") key: String,             // ← antes: @Query("key")
        @Path("nombre") ciudad: String,         // ← antes: @Query("q")
        @Header("days") dias: Int = 3           // ← antes: @Query("days")  (sin coma: es el último)
    ): RespuestaClimaDto                        // lang, aqi y alerts: BORRADOS
}
```

¡Importante! PokeAPI rechaza parámetros que no conoce

`.../pokemon/ditto` → 200 ✅ `.../pokemon/ditto?days=3` → **403** ❌

Si dejas los `@Query` de WeatherAPI, la app muestra **"El servicio no pudo responder (HTTP 403)"**.

• `lang`, `aqi` y `alerts` → **se borran**, porque `ClimaRepository` no los pasa.

• `key` y `dias` → **no se pueden borrar**, porque `ClimaRepository` llama `api.obtenerPronostico(key, consulta, 3)`. Se cambian a `@Header`, que manda el dato dentro de la petición y no en la URL. PokeAPI lo ignora sin quejarse.

De dónde sale

Comparas URLs y ves qué parte cambia: `pokemon/ditto`, `pokemon/pikachu`. La parte fija (`pokemon/`) se escribe tal cual, y la que cambia se reemplaza por un hueco `{nombre}`. El texto entre llaves tiene que ser igual al de `@Path("nombre")`.

• La parte que cambia está **dentro de la ruta** → `@Path`

• La parte que cambia está **después del** `?` → `@Query`

Paso 4 · Los DTOs (ClimaDtos.kt)

4.1 · Revisa que `UbicacionDto` esté como la original

Si la modificaste antes, devuélvela a su forma original. Está en `respaldo_weatherapi/ClimaDtos.kt`, líneas 12–19:

`UbicacionDto (original, NO se cambia)`

```
data class UbicacionDto(
    @SerializedName("name") val nombre: String,
    @SerializedName("region") val region: String,
    @SerializedName("country") val pais: String,
    @SerializedName("localtime") val horaLocal: String,
    @SerializedName("lat") val latitud: Double,
    @SerializedName("lon") val longitud: Double
)
```

> **Por qué no van ahí los datos del Pokémon**
> En WeatherAPI, `UbicacionDto` se llenaba con lo que había **dentro** de `"location": { }`. El JSON del Pokémon **no tiene** `"location"`: sus campos están en el primer nivel. Y el primer nivel del JSON siempre va en la clase que devuelve el servicio, que es `RespuestaClimaDto`.

4.2 · Modifica `RespuestaClimaDto` (al final del archivo)

`antes`

```
data class RespuestaClimaDto(
    @SerializedName("location") val ubicacion: UbicacionDto,
    @SerializedName("current") val actual: ClimaActualDto,
    @SerializedName("forecast") val pronostico: PronosticoDto
)
```

`después`

```
data class RespuestaClimaDto(
    @SerializedName("id") val id: Int,
    @SerializedName("name") val nombre: String,
    @SerializedName("height") val altura: Int,
    @SerializedName("weight") val peso: Int,
    @SerializedName("sprites") val sprites: SpritesDto
)
```

Cómo se escribe cada línea: usas el molde de tu archivo viejo, `@SerializedName("NOMBRE EN EL JSON") val NOMBRE QUE ELIGES: TIPO,`

Ves en el JSON	Comillas: copiado del JSON	`val` : lo eliges tú	Tipo: por la regla
`"id": 132`	`"id"`	`id`	`Int`
`"name": "ditto"`	`"name"`	`nombre`	`String`
`"height": 3`	`"height"`	`altura`	`Int`
`"weight": 40`	`"weight"`	`peso`	`Int`
`"sprites": { ... }`	`"sprites"`	`sprites`	llaves → clase nueva `SpritesDto`

> **Escribe el nombre del JSON exactamente igual**
> `"weigths"` ≠ `"weight"`. Si una letra no coincide, Gson no encuentra el dato y queda en 0 o null. Tampoco uses nombres de tipos como variable: `val Int` está mal, `val peso` está bien.

4.3 · Crea `SpritesDto`, copiando el molde de `CondicionDto`

`sprites` tiene llaves en el JSON, así que es un objeto dentro de otro y necesita su clase. Si no la creas, sale el error **Unresolved reference: SpritesDto**. Tu archivo ya tiene un caso igual: `"condition": { ... }` → `CondicionDto`. Copia ese molde y escríbela **justo arriba** de `RespuestaClimaDto`:

molde que ya tienes

```
data class CondicionDto(
    @SerializedName("text") val descripcion: String,
    ...
)
```

clase nueva, mismo molde

```
data class SpritesDto(
    @SerializedName("front_default") val imagen: String?
)
```

El nombre es el del JSON + `Dto`: `sprites` → `SpritesDto`, igual que `condition` → `CondicionDto`. Lleva `?` porque algunos Pokémon no tienen imagen y viene `null`.

Las demás clases (`CondicionDto`, `UbicacionDto`, `ClimaActualDto`, `ResumenDiaDto`, `DiaPronosticoDto`, `PronosticoDto`) **quedan igual**.

Paso 5 · El mapper (ClimaMapper.kt): solo la última función

Las 6 funciones de arriba **no se tocan**. Siguen compilando porque sus DTOs no cambiaron. Solo cambias la última:

antes (líneas 49–53)

```
fun RespuestaClimaDto.aEntidad(): RespuestaClima = RespuestaClima(
    location = ubicacion.aEntidad(),
    current = actual.aEntidad(),
    forecast = pronostico.aEntidad()
)
```

Ahora sale en rojo porque `ubicacion`, `actual` y `pronostico` ya no existen: los cambiaste en el Paso 4.2. En su lugar **armas cada entidad directamente**:

después

```
fun RespuestaClimaDto.aEntidad(): RespuestaClima = RespuestaClima(
    location = Ubicacion(                 // izquierda copiada de las líneas 13–20
        name = nombre,
        region = "Pokémon #$id",
        country = "",
        localtime = "",
        lat = altura / 10.0,
        lon = peso / 10.0
    ),
    current = ClimaActual(                // izquierda copiada de las líneas 22–31
        temp_c = altura / 10.0,
        feelslike_c = peso / 10.0,
        humidity = 0,
        wind_kph = 0.0,
        precip_mm = 0.0,
        uv = 0.0,
        last_updated = "",
        condition = Condicion(            // izquierda copiada de las líneas 7–11
            text = nombre,
            icon = sprites.imagen.orEmpty(),
            code = id
        )
    ),
    forecast = Pronostico(forecastday = emptyList())   // de las líneas 45–47
)
```

De dónde sale cada parte

```
temp_c   =   altura / 10.0
  │              │
  │              └─ DERECHA: un val de TU DTO (Paso 4.2), NUNCA el nombre del JSON
  └─ IZQUIERDA: copiado de las funciones de arriba (o Ctrl+clic en ClimaActual)
```

Parte	Sale de...
Lado izquierdo (`name =` , `temp_c =` ...)	Las funciones viejas del mismo archivo
Lado derecho (`nombre` , `altura` , `peso` , `id` , `sprites`)	Los `val` de tu `RespuestaClimaDto`
Lo que el Pokémon no tiene	`""` si es texto, `0` si es `Int` , `0.0` si es `Double`
`forecast`	`emptyList()` : una lista vacía, sin pronóstico

Cuando los tipos no coinciden (error "Type mismatch")

Ctrl + clic en la entidad muestra qué tipo pide cada campo. Si tu dato es de otro tipo, conviértelo:

Tienes	Pide	Cómo	Ejemplo
`Int`	`String`	Dentro de un texto con `$`	`region = "Pokémon #$id"`
`Int`	`Double`	`.toDouble()` o `/ 10.0`	`lat = altura / 10.0`
`Boolean`	`String`	Dentro de un texto	`"Default: $esDefault"`
`String?`	`String`	`.orEmpty()`	`icon = sprites.imagen.orEmpty()`
`Boolean`	`Double`	❌ No se puede	Elige otro dato

`/ 10.0` : PokeAPI da la altura en decímetros (3 → 0,3 m) y el peso en hectogramos (40 → 4 kg).

Paso 6 · Compilar y probar

- ☐ **Build → Rebuild Project.** No debe quedar nada en rojo.
- ☐ Ejecuta la app (▶).
- ☐ **Apaga el interruptor "Modo demostración".**
- ☐ Al abrir saldrá un error, porque busca "La Paz, Bolivia" y no es un Pokémon. Es normal.
- ☐ Escribe `ditto` **en minúsculas** y pulsa Buscar.
- ☐ Prueba también `pikachu` y `25`.

Lo que debes ver

```
┌──────────────────────────────┐
│ ditto                        │  ← nombre
│ Pokémon #132                 │  ← id
│ TEMPERATURA ACTUAL           │
│ 0,3°C          [imagen]      │  ← altura  +  sprites.front_default
│ ditto                        │  ← nombre
│ Sensación térmica de 4 °C    │  ← peso
└──────────────────────────────┘
```

Situación	Es normal porque...
`Ditto` con mayúscula da error 404	PokeAPI solo acepta minúsculas
Un Pokémon que no existe muestra "HTTP 404"	El repositorio solo conoce los errores de WeatherAPI
Dice "Datos: WeatherAPI.com" y "Humedad 0%"	La pantalla no se tocó, y esos casilleros no tienen dato
La pestaña Pronóstico está vacía	Pusimos `emptyList()`

Errores que puedes ver y cómo arreglarlos

Error	Causa	Solución
Unresolved reference: SpritesDto	Usas una clase que no creaste	Paso 4.3
Unresolved reference: ubicacion (o `actual`, `pronostico`)	La última función del mapper es la vieja	Paso 5
"El servicio no pudo responder (HTTP 403)"	Quedan `@Query` en el servicio: PokeAPI rechaza parámetros desconocidos	Paso 3: `@Header` y borrar los demás
Unresolved reference: Path / Header	Falta el import	`import retrofit2.http.Path` / `Header`
Type mismatch: inferred type is Int but String was expected	El tipo no coincide	Tabla de conversiones del Paso 5
Errores en `UbicacionDto.aEntidad()`	Modificaste `UbicacionDto`	Devuélvela a la original (Paso 4.1)
La app se cierra al abrir: *baseUrl must end in /*	Falta la `/` final	Paso 2
"Falta la clave de WeatherAPI..."	No existe `apikey.properties` o no hiciste Sync	Paso 1
Siempre muestra La Paz	El modo demo está encendido	Apaga el interruptor
"...datos que no se pudieron interpretar"	Un tipo del DTO no coincide con el JSON	Regla de tipos del Paso 0
Un dato sale siempre en 0	Nombre mal escrito en `@SerializedName`	Compáralo letra por letra con el JSON

Para practicar: agregar un dato tú mismo

Ejemplo: mostrar `"base_experience": 101` donde dice "Humedad".

1. **JSON:** `"base_experience": 101`. Es un número sin decimales, así que es `Int`.
2. **DTO** (dentro de `RespuestaClimaDto`, sin olvidar la coma de la línea anterior):
   `@SerializedName("base_experience") val experiencia: Int`
3. **Mapper:** reemplaza `humidity = 0,` por `humidity = experiencia,`

Con `"order": 214` en "Viento" sería: DTO → `@SerializedName("order") val orden: Int` y mapper → `wind_kph = orden.toDouble(),`

Resumen para clase

```
1. JSON en el navegador → veo los nombres y los tipos
2. apikey.properties    → WEATHER_API_KEY=pokeapi  + Sync
3. ApiClient            → baseUrl = lo que va antes de "pokemon/"
4. Service              → @GET("pokemon/{nombre}") + @Path("nombre")
```

```
                            key y days → @Header ; borrar lang, aqi, alerts
5. DTO                   → RespuestaClimaDto con los campos del JSON
                            llaves { } = clase nueva (molde de CondicionDto)
6. Mapper                → solo la última función:
                            izquierda = copiada de arriba, derecha = mis val
7. Rebuild → apagar demo → buscar "ditto"
```